Carbon-Aware Cloud Computing for AI in Marine Research: A Case Study on Sea Surface Temperature Prediction Along the South Florida Atlantic Coast

Bipin Chandra Chowdary Vadlamudi
Cloud Computing Research Paper
Florida Atlantic University
Spring 2026

***Abstract*—This paper presents a carbon-aware cloud computing case study for artificial intelligence in marine research. The project uses NOAA Optimum Interpolation Sea Surface Temperature data from 2021 to 2024 for the South Florida Atlantic Coast and trains multiple machine learning models to predict next-day sea surface temperature. The models include a persistence baseline, Ridge Regression, Random Forest, XGBoost, and a PyTorch neural network. Each model is evaluated using MAE, RMSE, R^2, training time, and estimated emissions. CodeCarbon is used to estimate local compute energy and emissions, and Google Cloud region carbon-intensity data is used to simulate how the same workload would behave across different cloud regions. The results show that Ridge Regression performs best, achieving an MAE of 0.0984°C, RMSE of 0.1336°C, and R^2 of 0.9958. It also improves RMSE by 7.92% compared with the persistence baseline. The cloud-region simulation shows that moving the best model's offline training workload from `us-east1` to `europe-north2` reduces estimated emissions by approximately 99.53%. The study shows that sustainable marine AI depends on both efficient model selection and carbon-aware cloud-region placement.**

***Index Terms*—carbon-aware cloud computing, marine AI, sea surface temperature, CodeCarbon, Google Cloud, Green AI, cloud sustainability**

I. Introduction

Artificial intelligence is becoming a normal part of environmental and marine research. It is used to process satellite observations, detect patterns in ocean variables, predict short-term environmental changes, and support decision-making in coastal regions. At first, this sounds like a purely positive development. More data, better models, faster predictions, and stronger monitoring systems. But there is another side to it. Every AI workload runs somewhere. It uses hardware. It consumes electricity. And depending on where that electricity comes from, the same model can have a very different carbon impact.

Cloud computing makes this problem more important because so much AI research now depends on cloud infrastructure. A researcher can store gridded satellite data in cloud storage, train a model on a virtual machine, use GPUs for deep learning, and deploy the final model through a cloud service. This flexibility is powerful. It removes many barriers that would otherwise make environmental AI projects difficult to run. Marine datasets are often large, spatial, and time-dependent, so cloud platforms are a natural fit for this type of work.

Sea surface temperature, or SST, is a good example. SST is not just a single measurement taken at one beach. It changes across latitude, longitude, season, weather patterns, ocean mixing, and coastal conditions. For South Florida, this matters because the Atlantic coastal region is connected to marine ecosystems, tourism, fisheries, coral stress, storm behavior, and long-term climate monitoring. A model that predicts next-day SST may look simple, but it still represents the kind of workflow that appears in many environmental AI systems: collect data, preprocess it, train models, evaluate accuracy, and run the workload on computing infrastructure.

The main idea of this paper is that AI models should not be evaluated only by accuracy. Accuracy matters, of course. A model that cannot predict well is not useful. But accuracy alone does not tell the whole story. Training time, energy consumption, model complexity, and cloud-region carbon intensity also matter. A model that is slightly more complex but uses much more energy may not be the best practical choice. In the same way, a cloud region that is convenient but carbon-intensive may not be the best place to run offline training jobs.

This paper applies carbon-aware cloud computing to a marine AI case study along the South Florida Atlantic Coast. The project uses daily NOAA sea surface temperature data from 2021 to 2024. The study region covers the Atlantic-side coastal waters from the Biscayne and Miami area toward West Palm Beach. This avoids the Gulf of Mexico and keeps the research focused on the side of South Florida where the major coastal cities are located.

The experiment trains five prediction approaches: a persistence baseline, Ridge Regression, Random Forest, XGBoost, and a PyTorch neural network. The target is next-day SST for each valid ocean grid cell. The models are compared using MAE, RMSE, and R^2, but the analysis does not stop there. CodeCarbon is used to estimate local training energy and

emissions. Then, the measured energy values are combined with Google Cloud region carbon-intensity data to simulate how the same workload would behave in different cloud regions.

This makes the project more than a theory paper. It is a small working case study. It starts with real marine data, creates machine-learning features, trains models, measures compute impact, and performs a carbon-aware cloud-region simulation. The goal is not to claim that this small model will solve ocean forecasting. It will not. The goal is to show a repeatable method for thinking about marine AI workloads in the cloud.

The research is guided by three main questions. First, how accurately can machine learning models predict next-day SST along the South Florida Atlantic Coast? Second, how do different model choices compare in accuracy, training time, and estimated emissions? Third, how much can estimated cloud emissions be reduced by selecting lower-carbon cloud regions for offline marine AI training workloads?

The results support a practical conclusion: bigger models are not always better. In this case study, Ridge Regression performs better than Random Forest, XGBoost, and the neural network. It also outperforms the persistence baseline, which is important because SST changes slowly from day to day. The neural network consumes the most training time and estimated emissions, but it performs the worst. That result is useful because it supports one of the central arguments of Green AI: model efficiency should be treated as part of model quality, not as an afterthought.

The cloud-region simulation adds the cloud computing layer. When the best model's measured energy use is simulated across selected Google Cloud regions, the estimated emissions differ sharply depending on the region. The lowest-carbon selected region is `europe-north2`, located in Stockholm, while `us-east1` has much higher grid carbon intensity. For offline training, moving the workload from `us-east1` to `europe-north2` reduces estimated emissions by about 99.53%. That does not mean every marine system should always run in Europe. Latency, cost, data movement, privacy, and service availability still matter. But for offline training and research experiments, carbon-aware region selection can be a strong sustainability strategy.

The rest of this paper is organized as follows. Section II reviews the background literature on cloud computing, marine SST data, AI energy impact, carbon-aware cloud computing, and emissions tracking tools. Section III will describe the dataset and methodology. Section IV will present the model results. Section V will discuss the cloud carbon simulation and practical tradeoffs. The paper ends with limitations, future work, and a conclusion.

II. Literature Review

A. Cloud Computing and AI Workloads

Cloud computing has changed how modern AI projects are built and executed. Instead of depending only on local hardware, researchers can use cloud infrastructure for storage, preprocessing, training, evaluation, and deployment. This is especially useful when the project involves large datasets or repeated experiments. A local machine may be enough for a small prototype, but cloud systems make it easier to scale a workflow when the data grows or when more computation is needed.

For marine and environmental research, this flexibility matters. Ocean datasets are often gridded, multidimensional, and time-dependent. A single dataset may include time, latitude, longitude, depth, temperature, salinity, current velocity, sea level, or other variables. Even when the present project uses only SST, the structure of the data still reflects the larger challenge of environmental computing. The data is not naturally arranged like a simple table. It must be subset, cleaned, transformed, and converted into a form that machine learning models can use.

Cloud platforms support this type of workflow because they combine storage, compute, networking, and software services. In a typical environmental AI pipeline, a researcher may download or access remote satellite data, store it in cloud storage, process it with Python notebooks, train models on virtual machines, and later expose the model through an API or dashboard. This ability to connect data processing and model deployment is one reason cloud computing is important for applied AI research.

However, cloud computing should not be treated as invisible infrastructure. Every cloud job runs in a physical data center. That data center has hardware, cooling systems, energy requirements, and a connection to an electrical grid. Because of this, the environmental impact of cloud computing depends on the workload and the region where it runs. The same model trained in two different regions can have different estimated emissions if the grid carbon intensity differs. This is where carbon-aware cloud computing becomes relevant.

B. Marine Data and Sea Surface Temperature

Sea surface temperature is one of the most important variables in ocean and coastal research. It helps describe the thermal condition of the ocean surface and influences marine ecosystems, air-sea exchange, weather patterns, and coastal environmental conditions. SST is also useful for machine learning because it has a strong temporal structure. Today's SST is usually close to tomorrow's SST, but it still changes enough to make prediction meaningful.

The present study uses NOAA OISST data because it is a reliable and widely used daily sea surface temperature product. OISST provides gridded SST estimates using a regular spatial grid, which makes it practical for regional machine learning experiments. The dataset is especially appropriate for this project because it supports both spatial analysis and time-series modeling. It also allows a focused regional subset to be extracted without needing to train on the entire global ocean.

The selected region in this project covers the South Florida Atlantic Coast from the Biscayne and Miami area toward West Palm Beach. This region was chosen deliberately. Earlier project planning considered wider areas, including the Gulf of Mexico, but the final study avoids the Gulf and focuses on

the Atlantic side where the South Florida coastal cities are actually located. That makes the case study more consistent with the research story. It also gives enough valid ocean grid cells for model training while keeping the project manageable.

Using a regional subset has another benefit. It keeps the work realistic for a semester research project. A global ocean forecasting model would require a much larger design, more advanced architectures, and more computational resources. That would shift the project away from cloud sustainability and toward ocean modeling itself. Here, the dataset is large enough to support real preprocessing, model training, and evaluation, but small enough to let the paper focus on the relationship between AI accuracy, energy use, and cloud-region emissions.

C. Environmental Impact of AI

AI research often celebrates larger models, higher accuracy, and more complex architectures. That makes sense in some settings. Complex models can solve problems that simple methods cannot. But complexity is not free. Training models consumes electricity, and repeated experiments multiply that cost. If a model is trained many times for tuning, comparison, or deployment, the environmental cost can grow quickly.

This has led to the idea of Green AI, where efficiency is considered alongside predictive performance. The point is not to avoid AI or to reject deep learning. The point is to evaluate AI more honestly. If two models perform similarly, but one model requires far less energy, then the lighter model may be the better engineering choice. In some cases, the simpler model may even perform better. That is exactly what happens in this project.

The environmental impact of AI also depends on where the computation happens. Training a model in a low-carbon region can produce lower estimated emissions than training the same model in a high-carbon region. This is especially relevant in cloud computing because cloud platforms allow workloads to be placed in different regions. For latency-sensitive applications, region selection may be constrained. For offline training, the choice is often more flexible.

This distinction matters for marine AI. A real-time coastal warning system may need low latency and regional availability, so it may not be able to run anywhere in the world. But offline model training is different. A training job can often run in a cleaner region, then the final model can be deployed closer to users if needed. This separation between training and deployment creates an opportunity for carbon-aware cloud design.

D. Carbon-Aware Cloud Computing

Carbon-aware cloud computing means making cloud decisions with carbon intensity in mind. Instead of choosing a region only based on convenience, cost, or default settings, the user also considers the carbon characteristics of that region. This can include grid carbon intensity, carbon-free energy percentage, time of day, workload flexibility, and whether the job is latency-sensitive.

In this project, carbon-aware cloud computing is applied through region selection. The local energy measured during model training is reused in a cloud simulation. The same workload energy is multiplied by the grid carbon intensity of selected Google Cloud regions. This allows the study to estimate how much the emissions would change if the workload were placed in different regions.

The basic calculation is:

$$E_{cloud} = \frac{P_{kWh} \times CI_{region}}{1000} \tag{1}$$

where E_{cloud} is estimated cloud emissions in kg CO_2eq, P_{kWh} is measured energy in kilowatt-hours, and CI_{region} is grid carbon intensity in gCO_2eq/kWh. The division by 1000 converts grams to kilograms.

This formula is simple, but it captures the core logic of the project. If energy stays constant and carbon intensity changes, estimated emissions change. That is the main reason carbon-aware cloud placement can matter. The model does not need to change. The dataset does not need to change. The only thing that changes is where the workload is executed.

E. Emissions Tracking Tools

To compare models fairly, this study needs a way to estimate energy and emissions from training. CodeCarbon is used for this purpose. It estimates compute-related energy use from hardware activity, including CPU, GPU, and memory usage. The tool then reports estimated emissions. In this project, each trained model is wrapped with CodeCarbon so that training time, energy, and emissions can be compared.

The use of CodeCarbon also makes the project easier to connect to cloud computing. Instead of guessing the workload energy, the project measures it during local training. The measured energy is then used as the input to the cloud-region simulation. This creates a clear link between the machine learning experiment and the cloud sustainability analysis.

Cloud Carbon Footprint is another tool related to this type of work. It is commonly used to estimate cloud emissions from cloud usage data. This project does not directly use a live cloud billing account, so it does not use Cloud Carbon Footprint as the main measurement tool. Still, the logic is similar: estimate energy use, connect it to a carbon factor, and use the result to make better cloud decisions.

The literature and tools point to the same larger idea. AI systems should be evaluated not only by what they predict, but also by what they consume. This does not mean every project must use the smallest possible model. It means researchers should understand the tradeoff. For this marine AI case study, that tradeoff becomes visible through the comparison of model accuracy, training time, estimated energy, and simulated cloud emissions.

III. Methodology

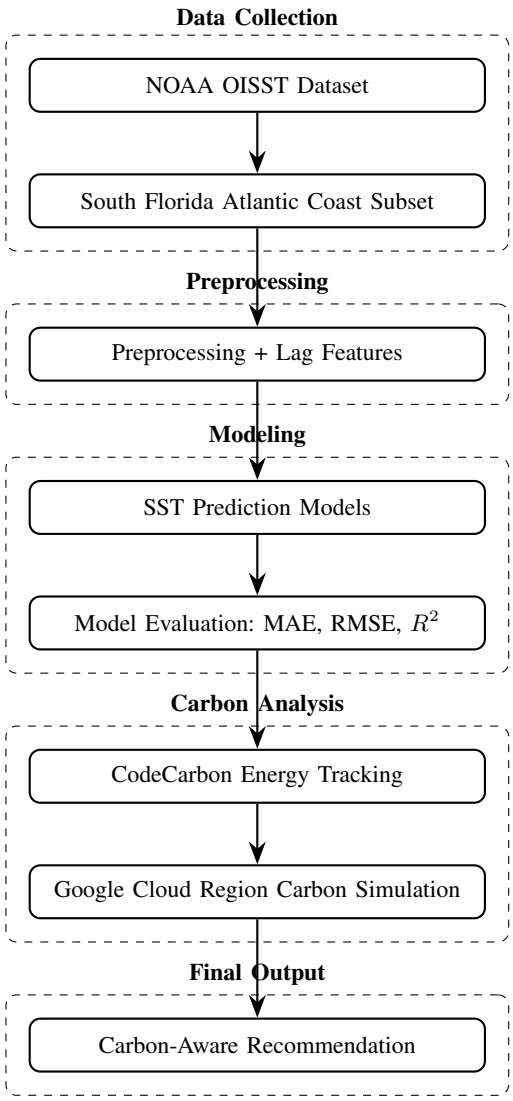

Fig. 1. Workflow of the proposed carbon-aware marine AI framework. The pipeline connects data collection, preprocessing, prediction modeling, emissions tracking, and cloud carbon simulation.

A. Project Design

This project was designed as a practical cloud computing experiment rather than a purely theoretical sustainability discussion. The main goal was to connect three parts of the problem: marine data, machine learning, and carbon-aware cloud execution. The workflow begins with real sea surface temperature data, converts it into a machine-learning-ready format, trains multiple prediction models, measures compute impact, and then simulates how the same workload would behave in different cloud regions.

The full pipeline is straightforward. First, daily NOAA OISST sea surface temperature data was collected for the South Florida Atlantic Coast. Next, invalid land and coastal grid cells were removed so that the model would train only on valid ocean points. After that, time-based and lag-based features were created. The processed dataset was then used to train multiple next-day SST prediction models. Finally, CodeCarbon was used to estimate training energy and emissions, and the measured energy values were combined with Google Cloud region carbon-intensity data to simulate cloud emissions.

This design was chosen because it represents a realistic research workflow. Marine researchers often train models offline before using them for analysis or deployment. Offline training is also the type of workload where carbon-aware scheduling makes the most sense. A real-time alert system may need to run near the users or near the data source. But training jobs have more flexibility. They can often be moved to a cleaner cloud region without affecting end users.

B. Study Area and Dataset

The study area focuses on the South Florida Atlantic Coast. The selected region extends from the Biscayne and Miami coastal area toward West Palm Beach. This region was chosen because it represents the Atlantic-side coastal waters near major South Florida cities, rather than the Gulf of Mexico. That choice matters. It keeps the case study tied to the actual coastal corridor of interest and avoids making the study region unnecessarily broad.

The dataset used in this project is NOAA OISST sea surface temperature data. The project used daily SST values from 2021 to 2024. The original regional subset contained 1,461 daily time steps, 7 latitude points, and 6 longitude points. This produced 42 grid cells per day before cleaning. Because the region includes coastline and land-adjacent grid cells, some missing values were expected. The raw subset contained 61,362 total values, with 14,610 missing values, or 23.81%.

A valid-ocean mask was created to remove grid cells that were mostly missing. Grid cells were kept only if they contained valid SST data for at least 90% of the selected time period. After this cleaning step, 32 valid ocean grid cells remained out of the original 42 grid cells. This was enough to support model training while still keeping the study region focused.

TABLE I
Dataset and Preprocessing Summary

Item	Value
Study region	South Florida Atlantic Coast
Time period	2021–2024
Raw daily time steps	1,461
Raw latitude points	7
Raw longitude points	6
Original grid cells	42
Valid ocean grid cells	32
Total raw values	61,362
Missing values	14,610
Missing percentage	23.81%
Final ML-ready rows	46,272
Final feature count	17

If the valid-ocean grid figure is available in the project folder, it should be included here because it visually supports

the preprocessing decision. It shows that the removed grid cells are mainly land or coastal cells where SST values are not consistently available.

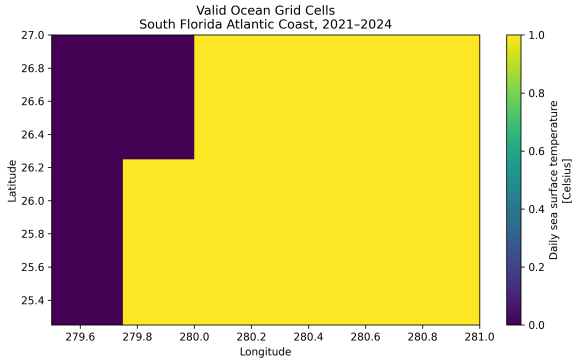

Fig. 2. Valid ocean grid cells retained after removing missing land and coastal cells from the South Florida Atlantic Coast SST subset.

C. Feature Engineering

After cleaning, the gridded SST data was converted into a tabular machine learning dataset. Each row represented one valid ocean grid cell on one day. The prediction target was the next-day SST for the same grid cell. This means the model was not trying to predict the entire ocean at once. It was learning how SST changes from one day to the next at each valid location in the selected coastal region.

The feature set included location features, time features, current SST, lagged SST values, and rolling statistics. Latitude and longitude were included because SST can vary slightly across the selected coastal box. Calendar features such as month and day of year were included because SST has strong seasonal behavior. Sine and cosine transformations were also used for month and day-of-year features so the model could understand seasonality as a cycle rather than as a simple linear number.

The lag features included SST from 1, 2, 3, 7, and 14 days before the target day. These features are important because SST is temporally persistent. In simple terms, today's ocean temperature tells us a lot about tomorrow's ocean temperature. A 7-day rolling mean and 7-day rolling standard deviation were also added to capture short-term local trends and variability.

The final model feature set contained 17 input variables:

- latitude and longitude,
- year, month, and day of year,
- sine and cosine seasonal encodings,
- current SST,
- SST lag values from 1, 2, 3, 7, and 14 days,
- 7-day rolling SST mean,
- 7-day rolling SST standard deviation.

The target variable was defined as:

$$y_t = SST_{t+1} \tag{2}$$

where y_t is the target value for a given row and SST_{t+1} is the next-day sea surface temperature for the same ocean grid cell.

D. Train-Test Split

The dataset was split by time instead of randomly. This is important. In time-dependent environmental data, random splitting can leak future information into the training process. To avoid that issue, the model was trained on data from January 15, 2021 to December 31, 2023 and tested on data from January 1, 2024 to December 30, 2024.

The training set contained 34,592 rows, and the test set contained 11,680 rows. Both the training and testing periods contained the same 32 valid ocean grid cells. This split allowed the models to learn from earlier years and then predict an unseen future year.

TABLE II
TRAINING AND TESTING SPLIT

Item	Value
Training period	2021-01-15 to 2023-12-31
Testing period	2024-01-01 to 2024-12-30
Training rows	34,592
Testing rows	11,680
Valid grid cells in training	32
Valid grid cells in testing	32

E. Models

Five models were evaluated in the experiment. The first was a persistence baseline. This baseline predicts tomorrow's SST as equal to today's SST. It is simple, but it is not weak. SST changes slowly, so a persistence baseline is a necessary comparison. If a model cannot beat persistence, then it may not be adding much value.

The second model was Ridge Regression. Ridge Regression is a linear model with regularization. It is lightweight, fast to train, and often performs well when the target has a strong linear relationship with the input features. Since next-day SST is strongly related to current and recent SST, Ridge Regression was expected to be competitive.

The third model was Random Forest. This model was included because it can capture nonlinear relationships and interactions between features. The fourth model was XGBoost, a gradient-boosted tree method often used for structured tabular data. The fifth model was a PyTorch feedforward neural network trained using GPU acceleration. This model was included to test whether a heavier neural approach would improve prediction accuracy enough to justify the extra compute.

The purpose of using multiple models was not just to find the best RMSE. The purpose was to compare accuracy and compute cost together. In a carbon-aware cloud project, that comparison is essential.

F. Evaluation Metrics

The models were evaluated using mean absolute error, root mean squared error, and coefficient of determination. MAE

gives the average absolute prediction error in degrees Celsius. RMSE penalizes larger errors more strongly, which makes it useful for environmental prediction. The R^2 score measures how much of the variance in the target variable is explained by the model.

The formulas are shown below:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{3}$$

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{4}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{5}$$

where y_i is the actual SST value, $\hat{y}_i$ is the predicted SST value, $\bar{y}$ is the mean observed SST, and n is the number of test observations.

Training time and estimated emissions were also recorded. CodeCarbon was used to estimate training emissions for the trained models. These values were later used in the cloud-region carbon simulation.

IV. Results

A. Model Performance

The model results show a clear outcome. Ridge Regression produced the best prediction performance on the 2024 test set. It achieved an MAE of 0.0984°C, an RMSE of 0.1336°C, and an R^2 score of 0.9958. This means the model tracked the next-day SST values very closely.

The persistence baseline was also strong, with an RMSE of 0.1451°C. That is expected because SST changes gradually from day to day. However, Ridge Regression still improved RMSE by 7.92% compared with the persistence baseline. This improvement is important because it shows that the model learned useful information from the additional lag, rolling, seasonal, and location features.

Random Forest and XGBoost did not outperform the baseline. Random Forest achieved an RMSE of 0.1529°C, while XGBoost achieved an RMSE of 0.1622°C. The PyTorch neural network performed the worst, with an RMSE of 0.9103°C. This result does not mean neural networks are bad. It means this particular feedforward neural network was not the right tool for this small structured SST prediction task.

TABLE III
Model performance comparison on the 2024 test set

Model	MAE	RMSE	R^2	RMSE Change
Ridge Regression	0.0984	0.1336	0.9958	+7.92%
Persistence Baseline	0.1083	0.1451	0.9950	0.00%
Random Forest	0.1119	0.1529	0.9944	-5.36%
XGBoost	0.1182	0.1622	0.9937	-11.76%
PyTorch Neural Network	0.7273	0.9103	0.8031	-527.19%

The RMSE comparison in Fig. 3 makes the result easy to see. Ridge Regression has the lowest error, while the neural network is far above the other models.

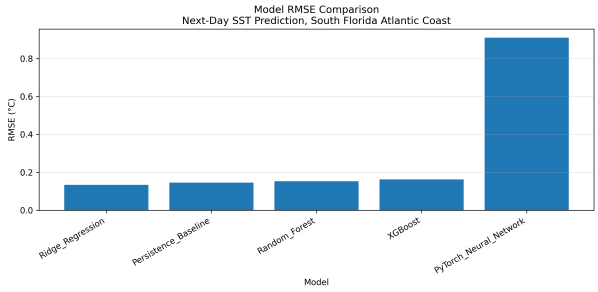

Fig. 3. RMSE comparison for next-day SST prediction models. Ridge Regression achieved the lowest RMSE.

B. Best Model Prediction Behavior

The actual-versus-predicted plot for Ridge Regression shows that the predicted values closely follow the observed next-day SST values. Most points fall near the diagonal line, meaning the model predictions are close to the actual measurements. This result supports the numerical performance shown in Table III.

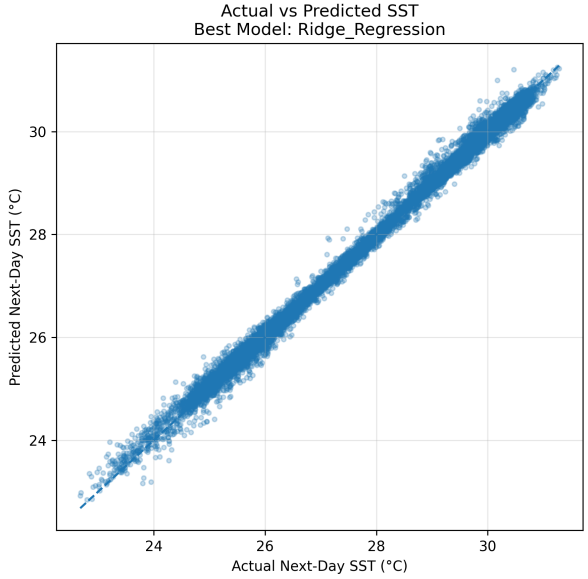

Fig. 4. Actual versus predicted next-day SST values for the best model, Ridge Regression.

The daily regional mean prediction plot also supports this conclusion. When predictions are averaged across the valid ocean grid cells, the Ridge Regression model follows the seasonal SST cycle very closely throughout 2024. The model captures the cooler winter period, the warming trend into summer, the warm peak around late summer, and the cooling period toward the end of the year.

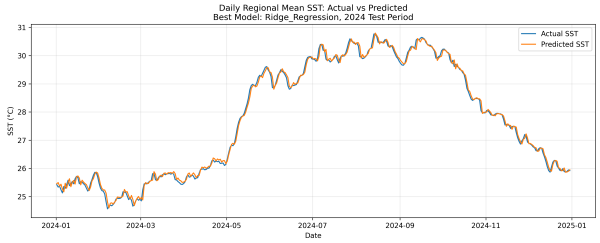

Fig. 5. Daily regional mean actual and predicted SST for the 2024 test period using Ridge Regression.

This is one of the strongest results in the project. It shows that the model is not only accurate point by point, but also follows the broader seasonal structure of the regional SST signal.

C. Training Time and Emissions

The training time and emissions results show why model efficiency matters. Ridge Regression trained almost instantly, with a training time of 0.0037 seconds and estimated emissions of 6.7169 mg CO_2eq. XGBoost also trained quickly, but with slightly higher emissions than Ridge Regression. Random Forest required more training time and more emissions than both Ridge Regression and XGBoost. The PyTorch neural network required the most training time and had the highest estimated emissions.

TABLE IV
TRAINING TIME AND ESTIMATED EMISSIONS

Model	Time (s)	kg CO_2eq	mg CO_2eq
Ridge Regression	0.0037	0.000007	6.7169
Random Forest	2.0843	0.000038	38.4482
XGBoost	0.2944	0.000010	10.4315
PyTorch Neural Network	10.0343	0.000104	104.3096

At first, these emissions values look very small. That is true. This is a small regional case study running on a local machine. But the pattern still matters. If a model is trained once, the emissions may be tiny. If it is trained thousands of times during hyperparameter tuning, repeated experiments, model updates, or multi-region deployment, the difference becomes more meaningful. The main value here is not the absolute carbon number from one small run. The value is the comparison between models and the repeatable method used to measure them.

D. Accuracy and Emissions Tradeoff

The most important result is the relationship between model accuracy and estimated emissions. Ridge Regression achieved the best accuracy while also having the lowest emissions among the trained models. This is the ideal outcome from a carbon-aware perspective. It means the best model was not only the most accurate, but also the most efficient.

The neural network shows the opposite pattern. It had the highest emissions and the worst accuracy. This is important because it challenges the assumption that a more complex model will automatically perform better. For this dataset and task, the simpler model was the better scientific and environmental choice.

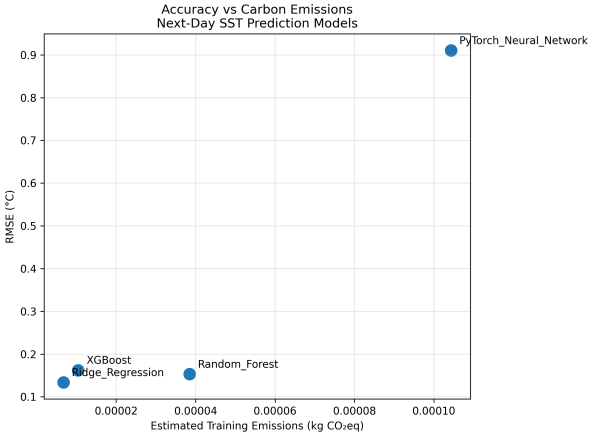

Fig. 6. Accuracy versus training emissions for the trained SST prediction models. Lower RMSE and lower emissions indicate a better accuracy-efficiency tradeoff.

This figure gives the main message of the model experiment. Carbon-aware AI is not only about running the same model in a cleaner cloud region. It also begins with choosing the right model for the problem. If a lightweight model performs better, then using a heavier model is hard to justify.

V. Cloud Carbon Simulation and Discussion

A. Purpose of the Cloud Simulation

The model training experiment shows which SST prediction model performs best and how much compute each model uses locally. But the main topic of this paper is cloud computing, so the next question is: what happens if the same workload is placed in different cloud regions?

This section answers that question through a cloud-region carbon simulation. The measured energy values from CodeCarbon were combined with 2024 Google Cloud region carbon-intensity data. The goal was not to claim that the model was physically trained in every cloud region. Instead, the simulation estimates how the same measured workload would translate into emissions if it were executed in selected Google Cloud regions with different grid carbon intensities.

This approach is useful because offline training jobs are often flexible. Unlike real-time applications, they do not always need to run close to the user. A marine research team could train a model in one region, store the trained model, and deploy it later wherever latency or operational needs require. That makes offline training a strong candidate for carbon-aware cloud placement.

B. Selected Cloud Regions

Seven Google Cloud regions were selected for the simulation. The list includes low-carbon regions, moderate-carbon U.S. regions, and a high-carbon comparison region. This makes the comparison more useful because it shows a wide

range of possible cloud-region outcomes rather than only comparing similar regions.

The selected regions were `europe-north2`, `northamerica-northeast1`, `europe-west6`, `us-west1`, `us-east4`, `us-east1`, and `asia-south1`. Among these, `europe-north2` in Stockholm had the lowest grid carbon intensity at 2.73 gCO_2eq/kWh and 100% carbon-free energy. In contrast, `us-east1` in South Carolina had a grid carbon intensity of 575.77 gCO_2eq/kWh and 31% carbon-free energy. The highest selected comparison region was `asia-south1` in Mumbai, with 678.76 gCO_2eq/kWh and 9% carbon-free energy.

TABLE V
SELECTED GOOGLE CLOUD REGIONS USED IN THE CARBON SIMULATION

Cloud Region	Location	CFE (%)	Carbon Intensity
`europe-north2`	Stockholm	100	2.73
`northamerica-northeast1`	Montreal	99	5.48
`europe-west6`	Zurich	98	15.05
`us-west1`	Oregon	87	79.23
`us-east4`	Northern Virginia	62	323.05
`us-east1`	South Carolina	31	575.77
`asia-south1`	Mumbai	9	678.76

The difference between these regions is large. This is exactly why cloud-region selection matters. If the same workload consumes the same amount of energy, but the grid carbon intensity changes from 575.77 gCO_2eq/kWh to 2.73 gCO_2eq/kWh, the estimated emissions will also change sharply.

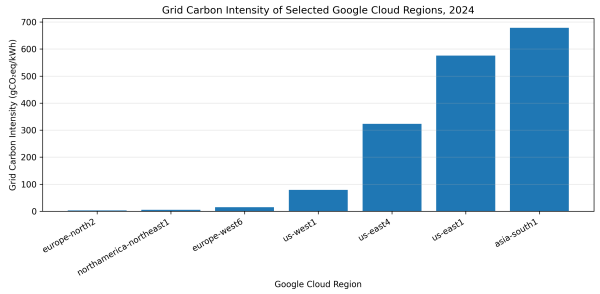

Fig. 7. Grid carbon intensity of selected Google Cloud regions in 2024. Lower values indicate lower estimated emissions per unit of energy consumed.

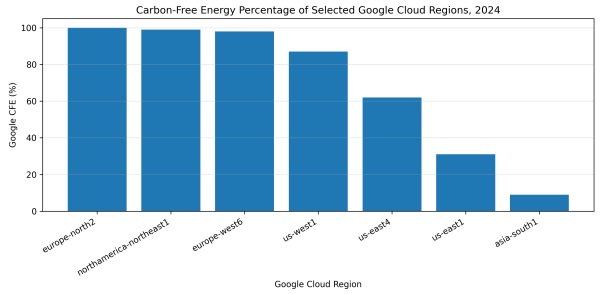

Fig. 8. Carbon-free energy percentage for selected Google Cloud regions in 2024.

C. Best Model Cloud-Region Results

Since Ridge Regression was the best model by RMSE, it was used as the main model for the cloud-region comparison. The measured energy consumption for Ridge Regression was 0.000014 kWh. That is a very small amount of energy because the model is lightweight and the dataset is a regional subset. Still, the result is useful because the same method can scale to larger models, larger datasets, and repeated training runs.

The simulation shows that the same Ridge Regression workload would produce very different estimated emissions depending on the selected region. In `europe-north2`, the simulated emissions were 3.947131e-08 kg CO_2eq. In `us-east1`, the simulated emissions were 0.000008 kg CO_2eq. In `asia-south1`, the simulated emissions were even higher.

TABLE VI
SIMULATED CLOUD EMISSIONS FOR THE BEST MODEL, RIDGE REGRESSION

Cloud Region	Location	Carbon Intensity	Emissions (mg CO_2eq)
`europe-north2`	Stockholm	2.73	0.0395
`northamerica-northeast1`	Montreal	5.48	0.0792
`europe-west6`	Zurich	15.05	0.2176
`us-west1`	Oregon	79.23	1.1455
`us-east4`	Northern Virginia	323.05	4.6708
`us-east1`	South Carolina	575.77	8.3247
`asia-south1`	Mumbai	678.76	9.8138

The difference is not small. Moving the best model's offline training workload from `us-east1` to `europe-north2` reduces estimated emissions by approximately 99.53%. This is the strongest cloud-computing result in the project. It shows that even when the model itself does not change, region choice can strongly affect estimated cloud emissions.

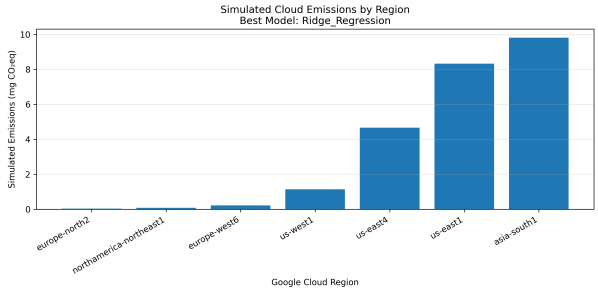

Fig. 9. Simulated cloud emissions for Ridge Regression across selected Google Cloud regions.

D. Carbon Savings Compared with `us-east1`

The region `us-east1` was used as the reference region because it is a common U.S. region and is relevant as a practical cloud option. Compared with this reference, every lower-carbon selected region produced savings. The largest savings came from `europe-north2`, followed by `northamerica-northeast1` and `europe-west6`.

This result is important, but it should be interpreted carefully. The lowest-carbon region is not always the best operational choice. For offline training, it may be reasonable to use

a faraway low-carbon region because latency is not a major issue. For real-time marine monitoring, emergency response, or interactive dashboards, latency and data transfer may matter more. In those cases, a closer region such as `us-west1` could be a practical compromise. It is not as clean as Stockholm or Montreal, but it is much cleaner than `us-east1` in the selected data.

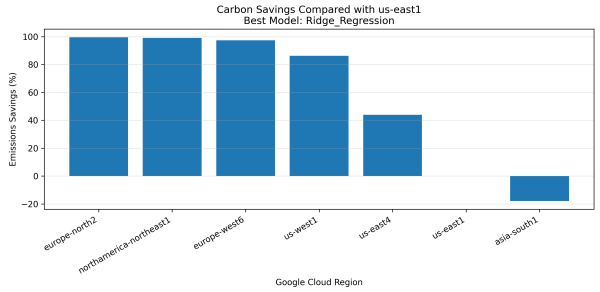

Fig. 10. Carbon savings for Ridge Regression compared with the `us-east1` reference region.

E. Accuracy Versus Simulated Cloud Emissions

The cloud simulation also supports the earlier model-selection result. Ridge Regression had the best accuracy and one of the lowest simulated cloud emissions values. The neural network had the worst accuracy and the highest measured energy. This means the project's main sustainability finding has two layers.

First, model choice matters. A simple model can be more accurate and more efficient than a heavier model. Second, cloud-region choice matters. Once the energy use is known, the same workload can have very different emissions depending on where it runs. Carbon-aware marine AI needs both parts. It needs efficient model design and thoughtful cloud placement.

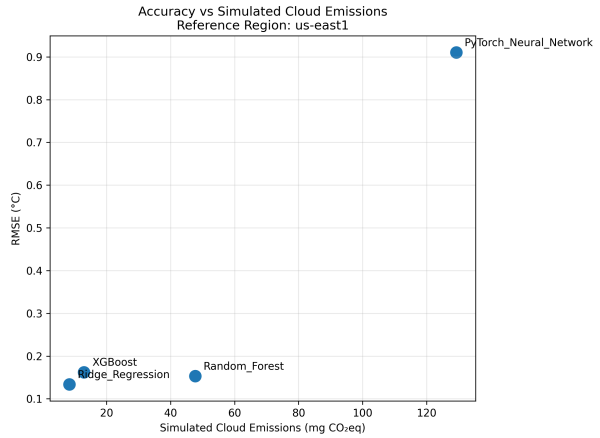

Fig. 11. Accuracy versus simulated cloud emissions for trained models using `us-east1` as the reference cloud region.

F. Scaled Research-Lab Scenario

A fair question is: if the measured emissions are so small, why does this matter? The answer is scale. A single lightweight training run does not produce much carbon. But research workflows are rarely only one training run. Models are retrained, compared, tuned, updated, and sometimes executed across many regions or many projects. A small difference can become more meaningful when repeated.

To illustrate this, the project also includes a scaled scenario of 10,000 repeated training runs. This is not presented as the actual measured footprint of the project. It is a scenario analysis. It shows how repeated experiments can magnify the effect of cloud-region selection.

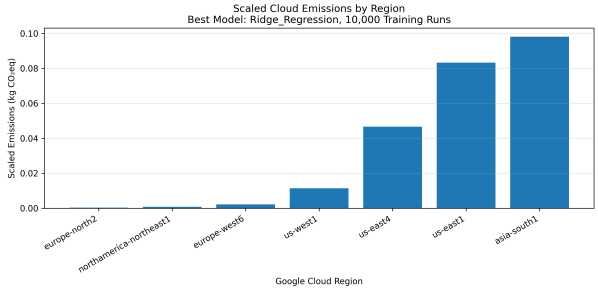

Fig. 12. Scaled cloud emissions for 10,000 Ridge Regression training runs across selected Google Cloud regions.

The scaled scenario makes the practical lesson clearer. If a marine research lab runs repeated offline experiments, region choice can matter. It may not be the first concern for a one-time classroom model, but it becomes more important as workloads grow.

G. Discussion

The results support a simple but useful argument: carbon-aware cloud computing should start before cloud deployment. It starts when the researcher chooses the model. In this study, Ridge Regression was more accurate than the heavier models and more efficient to train. That means the best environmental decision was not only to move the workload to a cleaner cloud region. It was also to avoid unnecessary model complexity.

This is where the result becomes practical. Many AI projects automatically treat deep learning as the advanced option. But for this dataset, the neural network did not help. It used more time, more energy, and produced worse predictions. A simpler model matched the structure of the problem better because next-day SST is strongly related to recent SST values and seasonal patterns.

The cloud simulation adds a second layer. Once the model is selected, the region matters. A flexible offline training job can be placed in a cleaner region without changing the model's prediction quality. That is the strength of carbon-aware cloud computing. It does not require sacrificing accuracy. It asks whether the same computational work can be done in a cleaner way.

At the same time, this result should not be overstated. A low-carbon region may not always be the correct choice. If a system needs low-latency inference for South Florida users, training in Stockholm and serving from Stockholm may

not make sense. Data transfer costs, privacy rules, service availability, and operational reliability may influence the final decision. A mature carbon-aware strategy should consider those constraints, not ignore them.

For this project, the strongest recommendation is clear: use lightweight models when they perform well, and use low-carbon regions for flexible offline training. That combination gives the best balance between predictive performance and environmental responsibility.

VI. Limitations and Future Work

A. Limitations

This study has several limitations. The first limitation is the size of the dataset. The project uses a regional subset of NOAA OISST data for the South Florida Atlantic Coast. This is appropriate for a focused semester case study, but it is not the same as a full operational ocean forecasting system. A larger study could include more ocean regions, longer time periods, and additional variables such as wind, currents, salinity, sea level, or atmospheric conditions.

The second limitation is the prediction task. This project predicts next-day SST. That is useful for a clean machine learning experiment, but it is a short forecasting horizon. Longer-term forecasts would be more difficult and would likely require more advanced spatiotemporal models. The strong performance of Ridge Regression in this study should therefore be understood in context. It works well for this short-horizon, feature-engineered SST task. It may not be the best model for every marine prediction problem.

The third limitation is the neural network design. The PyTorch model used in this project was a feedforward neural network. It was not a specialized ocean forecasting model. It did not use convolutional layers, recurrent layers, attention mechanisms, or ConvLSTM-style spatiotemporal modeling. A more advanced neural architecture might perform better, but it would also require more careful tuning and more compute. That would be a useful future extension.

The fourth limitation is the carbon measurement method. CodeCarbon estimates local training energy and emissions, but it does not directly measure every detail of the hardware and electrical system. The cloud analysis is also a simulation, not a direct cloud deployment measurement. The study uses measured local energy and multiplies it by cloud-region carbon intensity. This is a reasonable method for comparing regions, but it is still an estimate.

The fifth limitation is that the cloud-region carbon data uses annual values. Real carbon intensity changes by hour, season, and grid condition. A more advanced carbon-aware scheduler would use hourly or forecasted carbon intensity, not only yearly regional values. That would allow the workload to be shifted not only by region, but also by time.

Finally, the study does not include cost, latency, or data governance as optimization variables. These factors matter in real cloud systems. A region with lower carbon intensity may be farther away, more expensive for data transfer, or less suitable for a production system. This project focuses mainly on the accuracy-energy-emissions relationship.

B. Future Work

Future work can extend this project in several ways. One improvement would be to use a larger marine dataset. The study could cover a wider coastal region, include the Florida Keys, or compare multiple regions such as South Florida, the Gulf Stream, and the Caribbean. A longer historical period could also make the model more robust and allow better analysis of seasonal and interannual variation.

Another extension would be to add more environmental variables. SST is important, but ocean systems are not controlled by SST alone. Wind speed, air temperature, pressure, chlorophyll, currents, and sea surface height could all improve prediction depending on the task. A multi-variable dataset would make the modeling problem more realistic.

The modeling approach could also be expanded. Future work could compare ConvLSTM, temporal convolutional networks, transformers, graph neural networks, or physics-informed machine learning. These models may capture spatial and temporal dependencies more effectively than the simple models used here. The key question would be whether their accuracy improvement is large enough to justify the additional energy cost.

The carbon-aware cloud analysis could also become more realistic. Instead of using only annual region carbon intensity values, future work could use hourly carbon-intensity data and build a scheduler that recommends the best time and region for training. This would turn the project from region-aware placement into full carbon-aware scheduling.

Another useful extension would be to run the workload directly on a cloud platform. That would allow the study to compare local CodeCarbon estimates with actual cloud usage metrics, cloud cost, data transfer time, and region-specific performance. It would also make the project more realistic from a cloud engineering point of view.

Finally, the project could include a deployment scenario. For example, the model could be trained in a low-carbon region and then deployed closer to South Florida users for inference. This would separate the offline training decision from the real-time serving decision. That kind of architecture may be the most practical option for real marine AI systems.

VII. Code and Data Availability

The project code, notebooks, processed outputs, and supporting materials are available in a GitHub repository:

https://github.com/BipinChowdary/carbon_aware_marine_ai

The repository includes the workflow used for downloading and preprocessing the NOAA OISST data, training the SST prediction models, tracking emissions with CodeCarbon, simulating cloud-region carbon impact, and generating the figures used in this paper. Providing the repository helps make the work more transparent and reproducible, since the main experimental steps can be reviewed and rerun from the uploaded notebooks and scripts.

VIII. Conclusion

This paper presented a carbon-aware cloud computing case study for artificial intelligence in marine research. The project focused on next-day sea surface temperature prediction along the South Florida Atlantic Coast, using NOAA OISST data from 2021 to 2024. The main purpose was not only to build an accurate prediction model, but to study how model choice and cloud-region placement affect the environmental impact of an AI workload.

The results show that Ridge Regression was the best model for this task. It achieved an MAE of 0.0984°C, an RMSE of 0.1336°C, and an R^2 value of 0.9958. It also improved RMSE by 7.92% compared with the persistence baseline. That matters because the persistence baseline was already strong. Sea surface temperature changes slowly from day to day, so simply predicting tomorrow's SST as today's SST is not a weak comparison. Ridge Regression still performed better, which shows that the lag, rolling, seasonal, and location features added useful predictive information.

The model comparison also showed that a more complex model is not automatically better. Random Forest, XGBoost, and the PyTorch neural network did not outperform Ridge Regression. In fact, the neural network had the highest training time, the highest estimated emissions, and the weakest prediction accuracy. This result is one of the most important findings of the paper. It supports the idea that sustainable AI starts with choosing the right model for the problem. If a simple model performs better and consumes less compute, then using a heavier model is difficult to justify.

The cloud carbon simulation added another layer to the analysis. Using the measured energy from CodeCarbon and Google Cloud region carbon-intensity data, the study estimated how the same workload would behave across selected cloud regions. For the best model, moving the offline training workload from `us-east1` to `europe-north2` reduced estimated emissions by approximately 99.53%. This result shows that cloud-region selection can make a large difference when workloads are flexible.

At the same time, the conclusion should be realistic. The lowest-carbon region is not always the best practical region for every system. A real-time coastal monitoring system may need lower latency, regional availability, or specific compliance requirements. But offline model training is different. It can often be moved to a cleaner region without affecting end users. This makes offline training a strong candidate for carbon-aware cloud placement.

Overall, the project shows that carbon-aware cloud computing is not just a theoretical sustainability idea. It can be applied through a practical workflow: collect real scientific data, train models, measure energy, compare accuracy and emissions, and simulate cloud-region placement. For marine AI research, this approach can help researchers make better decisions before scaling their workloads. The strongest lesson is simple: use efficient models when they work, and place flexible cloud workloads in lower-carbon regions when possible.

Future marine AI systems can build on this work by using larger datasets, adding more ocean variables, testing advanced spatiotemporal models, using hourly carbon-intensity forecasts, and running direct cloud deployment experiments. Even then, the central idea will remain the same. AI should not be judged only by what it predicts. It should also be judged by what it consumes.

References

[1] NOAA National Centers for Environmental Information, "Optimum Interpolation Sea Surface Temperature (OISST)." [Online]. Available: https://www.ncei.noaa.gov/products/optimum-interpolation-sst. Accessed: May 5, 2026.

[2] NOAA National Centers for Environmental Information, "NOAA 0.25-degree Daily Optimum Interpolation Sea Surface Temperature (OISST), Version 2.1." [Online]. Available: https://www.ncei.noaa.gov/access/metadata/landing-page/bin/iso?id=gov.noaa.ncdc:C01606. Accessed: May 5, 2026.

[3] B. Huang, C. Liu, V. Banzon, E. Freeman, G. Graham, B. Hankins, T. Smith, and H.-M. Zhang, "Improvements of the Daily Optimum Interpolation Sea Surface Temperature (DOISST) Version 2.1," *Journal of Climate*, vol. 34, no. 8, pp. 2923–2939, 2021.

[4] Google Cloud, "Carbon free energy for Google Cloud regions." [Online]. Available: https://cloud.google.com/sustainability/region-carbon. Accessed: May 5, 2026.

[5] Google Cloud Platform, "Region Carbon Info: 2024 Google Cloud Region Carbon Data." [Online]. Available: https://github.com/GoogleCloudPlatform/region-carbon-info/blob/main/data/yearly/2024.csv. Accessed: May 5, 2026.

[6] CodeCarbon, "CodeCarbon: Track emissions from compute." [Online]. Available: https://github.com/mlco2/codecarbon. Accessed: May 5, 2026.

[7] Cloud Carbon Footprint, "Cloud Carbon Footprint." [Online]. Available: https://www.cloudcarbonfootprint.org/. Accessed: May 5, 2026.

[8] E. Strubell, A. Ganesh, and A. McCallum, "Energy and Policy Considerations for Deep Learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 3645–3650.

[9] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.

[10] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "Carbon Emissions and Large Neural Network Training," *arXiv preprint arXiv:2104.10350*, 2021.